
CS 120: Computer Networks Projects Report

From Acoustic Link to Internet Services

Renyi Yang
2023533030
yangry2023@shanghaitech.edu.cn

Zheng Yan
2023533083
yanzheng2023@shanghaitech.edu.cn

Abstract

This report details the design and implementation of a full-stack acoustic network system, evolved through four distinct projects. **Project 1** established a software-defined acoustic modem using BPSK modulation. **Project 2** focused on the Data Link Layer, implementing a high-performance Go-based MAC with CSMA/CA and a specialized Gap-Start Detection mechanism to withstand adversarial jamming. **Project 3** bridged the acoustic link to the host OS, utilizing a Loopback Adapter to intercept terminal traffic (e.g., Ping), and implementing a Router, NAT, and IP Fragmentation. Finally, **Project 4** implemented Transport Layer protocols, enabling UDP-based DNS resolution and reliable TCP-based HTTP requests over sound waves.

1 Introduction

Communication over acoustic channels is severely constrained by low bandwidth and environmental interference. This coursework mimics the evolution of the TCP/IP stack. We started by modulating bits over air (Project 1), secured the link against jamming and managed multi-node access (Project 2), routed IP packets to the real world (Project 3), and finally supported complex application-layer services (Project 4).

2 Project 1: Acoustic Link

The initial phase focused on establishing a basic digital link between two computers using standard audio hardware.

2.1 Modulation and Signal Processing

We implemented **Binary Phase Shift Keying (BPSK)** on a carrier frequency of $f = 10$ kHz.

- **Sender:** Binary data is modulated where '0' corresponds to $\sin(2\pi ft)$ and '1' to $-\sin(2\pi ft)$.
- **Receiver:** The incoming signal is processed through a high-pass Butterworth filter (> 1 kHz) to eliminate low-frequency ambient noise before demodulation.

2.2 Frame Synchronization

To achieve robust frame alignment, we employed a **Chirp Preamble**, a signal sweeping from 2.5 kHz to 8 kHz. The receiver operates as a state machine:

1. **Sync State:** Computes cross-correlation with the reference chirp. A peak indicates the start of a frame.

2. **Decoding State:** Integrates the product of the received signal and the reference carrier over each bit period (0.001s for 1000 Baud).

3 Project 2: Manage Multiple Access & Jamming Resilience

In the second project, we migrated the system to **Go** to leverage its concurrency model for real-time processing. The focus was to support communication between 4 nodes sharing a single acoustic channel while maintaining robustness against interference.

3.1 Physical Layer Enhancements

We upgraded the modulation to **NRZ** line coding with a baud rate of 12 kbps. The frame structure was standardized with a 56-bit header containing *Dest*, *Src*, *Type*, *FrameID*, and a CRC-8 checksum to ensure data integrity.

3.2 Reliable Transmission: Sliding Window & ACKs

Due to the high latency of acoustic propagation, a simple Stop-and-Wait protocol would severely limit throughput. We implemented a **Sliding Window** protocol with **Selective Repeat** ARQ to optimize bandwidth usage.

- **Window Logic:** The sender maintains a window of size $N = 10$ frames. It is allowed to transmit up to 10 unacknowledged frames continuously. The window slides forward only when the oldest frame in the buffer is acknowledged.
- **ACK Mechanism:** Upon successfully receiving a data frame and verifying the CRC, the receiver waits for a **SIFS** (Short Interframe Space) of 10 ms before sending an ACK. Crucially, the ACK transmission is also gated by the *Gap-Start Detector*; if the channel is predicted to be noisy, the ACK is deferred to prevent corruption.
- **Retransmission:** If an ACK is not received within the *RTTTimeout* (300 ms), the sender retransmits the specific frame. We set a limit of *MaxRetries* (8 attempts) before dropping the packet.

3.3 Multi-Node Contention: Exponential Backoff

To manage collisions among 4 nodes, we implemented a CSMA/CA algorithm with a "Freezing" backoff mechanism. The system defines a **Slot Time** of 9 ms.

1. **Carrier Sense:** Before transmitting, a node listens to the channel. If the channel is **IDLE** for a duration of **DIFS** (28 ms), it may proceed.
2. **Backoff Calculation:** If the channel is **BUSY**, the node enters a backoff state. It selects a random number of slots from a Contention Window (CW):

$$\text{Backoff Slots} = \text{Random}(0, CW), \quad \text{where } CW \in [3, 31] \quad (1)$$

The CW doubles upon each collision (Exponential Backoff) to reduce congestion, reset to 3 upon success.

3. **Freezing Mechanism:** During the backoff countdown, the node continues to monitor the channel. If the channel becomes **BUSY** (e.g., another node or jammer starts transmitting), the countdown is **frozen**. It resumes only after the channel has been **IDLE** for **DIFS** again.

3.4 Jamming Resilience: Gap-Start Detection

Standard carrier sensing fails in jamming scenarios (50–100 ms noise bursts) because nodes often transmit at the tail end of a quiet gap. We implemented a **Gap-Start Detector** state machine.

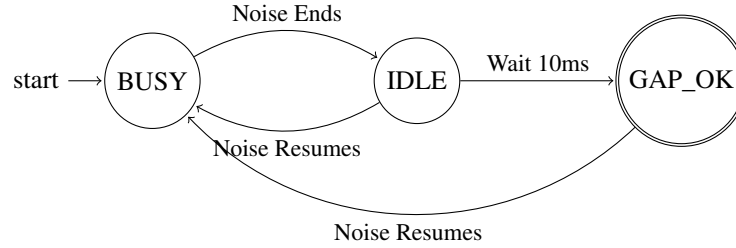


Figure 1: Gap-Start Detection State Machine.

The detector requires the channel to be clear for a strict validation threshold (10 ms) *after* a busy state. This ensures transmission begins early in the quiet window, leaving sufficient time (≈ 71 ms) to complete the frame.

4 Project 3: To the Internet

Project 3 integrated the acoustic network with the host OS. The primary goal was to allow standard terminal commands (like ping) to function seamlessly over the acoustic link.

4.1 Interaction with Host OS (Ping Workflow)

To intercept IP packets from the OS kernel, we installed the **Microsoft KM-TEST Loopback Adapter** and utilized the ‘gopacket’ library. The workflow for a terminal ping operation (e.g., ping 172.18.0.2) is implemented as follows:

1. **Packet Capture:** The OS routes the ICMP request to the loopback interface. Our program, listening on this interface, captures the raw Ethernet frame.
2. **Virtual ARP Responder:** Since no physical hardware exists, the OS issues ARP requests to find the MAC address of the target IP. We implemented a virtual ARP handler that intercepts these requests and replies with a virtual MAC address, "tricking" the OS into sending the actual IP payload.
3. **Encapsulation & Transmission:** Once the IP packet is captured, it is serialized into acoustic frames and transmitted via the speaker.
4. **Injection:** Upon receiving an acoustic reply, the node reconstructs the IP packet and **injects** it back into the loopback interface. The OS kernel recognizes this as a valid incoming packet, and the terminal displays the ping reply (e.g., "Reply from 172.18.0.2: bytes=32 time=XXms").

4.2 Router, NAT and Fragmentation

Node 2 acts as a gateway bridging the acoustic network and the physical Wi-Fi.

- **NAT Implementation:** Since ICMP headers lack ports, we mapped the **ICMP Identifier** to track ping sessions. The gateway translates ‘(Internal IP, Original ID)’ to a unique ‘New ID’ for outbound traffic, and reverses the mapping for inbound replies.
- **IP Fragmentation:** The acoustic link has a small Maximum Transmission Unit (MTU) of 100 bytes ("AethernetMTU"). Standard Ethernet frames (1500 bytes) are sliced at the sender and reassembled at the receiver, making the link transparent to upper layers.

5 Project 4: Transport Layer Services

The final project implemented custom userspace UDP and TCP protocols to support real-world applications.

5.1 UDP and DNS Resolution

We developed a complete DNS client that constructs standard DNS queries and encapsulates them in UDP datagrams (Port 53).

- **Workflow:** A query for ‘www.example.com’ is sent via acoustic link to the gateway.
- **Forwarding:** The gateway forwards the UDP packet to upstream resolvers (e.g., 1.1.1.1).
- **Result:** The resolved IP (e.g., 93.184.216.34) is parsed and returned to the application.

5.2 TCP and HTTP Implementation

To support web browsing (e.g., ‘curl’), we built a reliable TCP client with a full state machine.

5.2.1 TCP State Machine

The implementation manages the connection lifecycle:

- CLOSED → SYN_SENT: Initiating the 3-way handshake (sending SYN).
- SYN_SENT → ESTABLISHED: Receiving SYN-ACK, sending ACK.
- ESTABLISHED → FIN_WAIT: Closing the connection gracefully.

5.2.2 HTTP Data Transfer

Once connected, the system sends valid HTTP/1.0 GET requests using the PSH flag. The TCP layer ensures reliability via Sequence/Acknowledgment numbers and retransmissions. Incoming TCP segments containing HTML content are buffered and reassembled into a complete HTTP response, allowing the user to browse the web over sound.

6 Conclusion

We have successfully designed and implemented a full-stack network over sound. From the physical modulation of bits to the complexity of TCP state machines, the "Aethernet" system demonstrates that standard internet protocols can be adapted to function effectively even over highly constrained acoustic channels.

7 Suggestions for Future Iterations

Based on our experience, we strongly recommend providing students with a diagnostic tool in Project 1 to verify hardware functionality before diving into code implementation.

7.1 Hardware Verification Tool

We suggest including a pre-built **signal detection script** (either as a Jupyter notebook or MATLAB script) that can:

- Capture audio input from the microphone
- Display the spectrogram of received signals
- Measure basic signal quality metrics (SNR, peak amplitude)

7.2 Lessons Learned

During Project 1, our team spent a considerable amount of time debugging our modulation and demodulation code, suspecting software bugs. In reality, the root cause was **hardware-related issues**—improper audio driver configuration, incorrect JACK routing, or microphone sensitivity problems. A simple diagnostic tool would have immediately revealed that no acoustic signal was being received at all, saving hours of futile code debugging. This hardware verification step should be the first checkpoint before students proceed with protocol implementation.

A Appendix: Environment Setup

To replicate the experiment, the JACK Audio Server must be configured correctly.

A.1 Start JACK Server

Run the following command in PowerShell to start the audio backend with ASIO support:

```
cd "C:\Program Files\JACK2\  
.\jackd.exe -S -X winmme -dportaudio -d"ASIO::ASIO4ALL v2" -r48000 -p128
```

Parameters:

- -r48000: Sample rate of 48kHz.
- -p128: Buffer size of 128 samples (adjust for latency/stability trade-off).